



EXPRESS APPLICATION GENERATOR

- Use the application generator tool, express-generator, to quickly create an application skeleton.

```
npm i -g express-generator
```

- For example, the following creates an Express app named myapp. The app will be created in a folder named myapp in the current working directory and the view engine will be set to EJB.

```
express --view=pug myapp
```

```
create : myapp\  
create : myapp\public\  
create : myapp\public\javascripts\  
create : myapp\public\images\  
create : myapp\public\stylesheets\  
create : myapp\public\stylesheets\style.css  
create : myapp\routes\  
create : myapp\routes\index.js  
create : myapp\routes\users.js  
create : myapp\views\  
create : myapp\views\error.ejs  
create : myapp\views\index.ejs  
create : myapp\app.js  
create : myapp\package.json  
create : myapp\bin\  
create : myapp\bin\www
```

```
change directory:  
> cd myapp
```

```
install dependencies:  
> npm install
```

```
run the app:  
> SET DEBUG=myapp:* & npm start
```

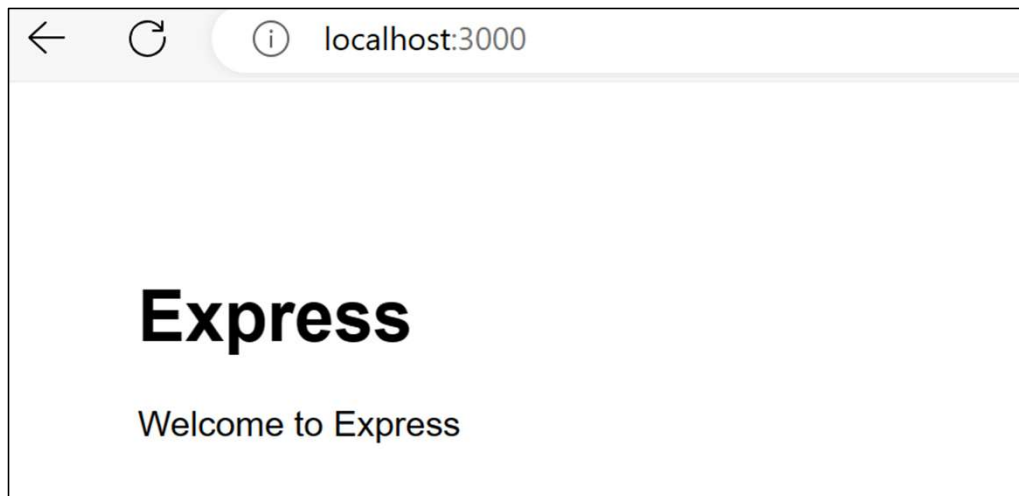
EXPRESS APPLICATION GENERATOR

- Then install dependencies:

```
npm install
```

```
set DEBUG=myapp:*
```

```
npm start
```

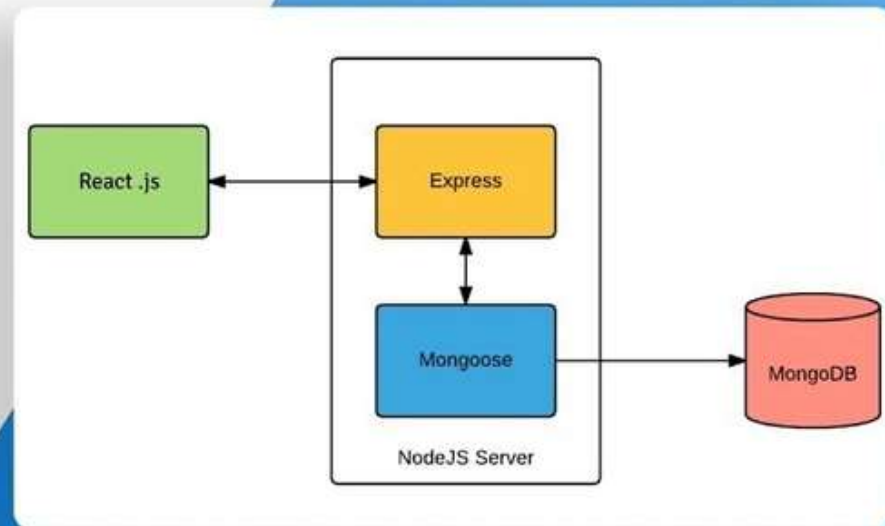


```
✓ MYAPP
  ✓ bin
    ≡ www
  ✓ public
    > images
    > javascripts
    > stylesheets
  ✓ routes
    JS index.js
    JS users.js
  ✓ views
    <> error.ejs
    <> index.ejs
  JS app.js
  {} package.json
```

Task Management System using Node and Express.js

NODEJS

Create Express Router & MVC

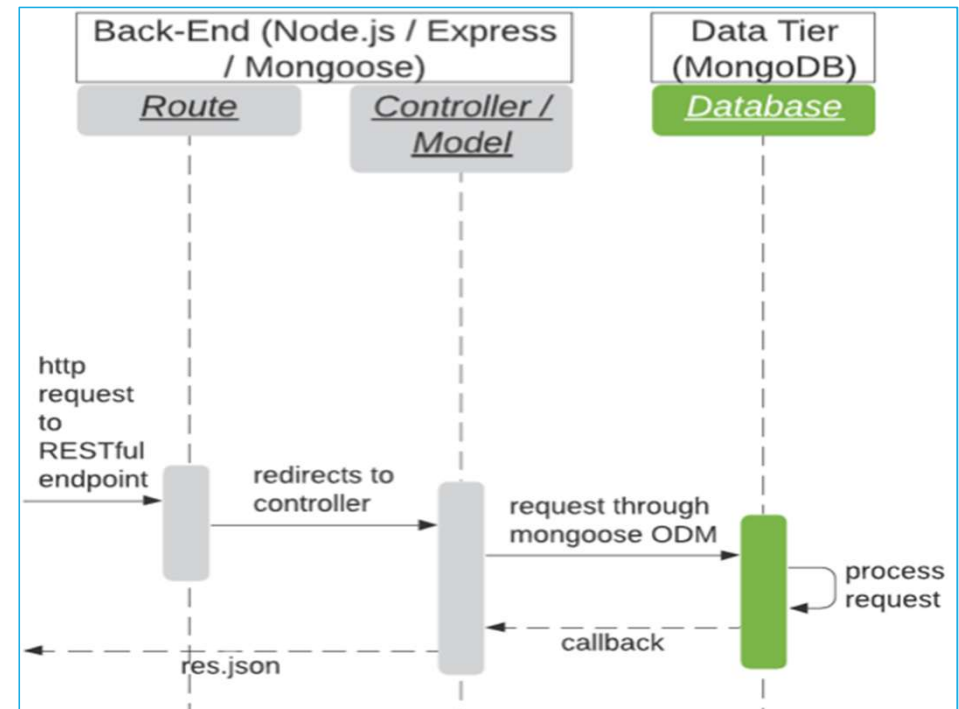
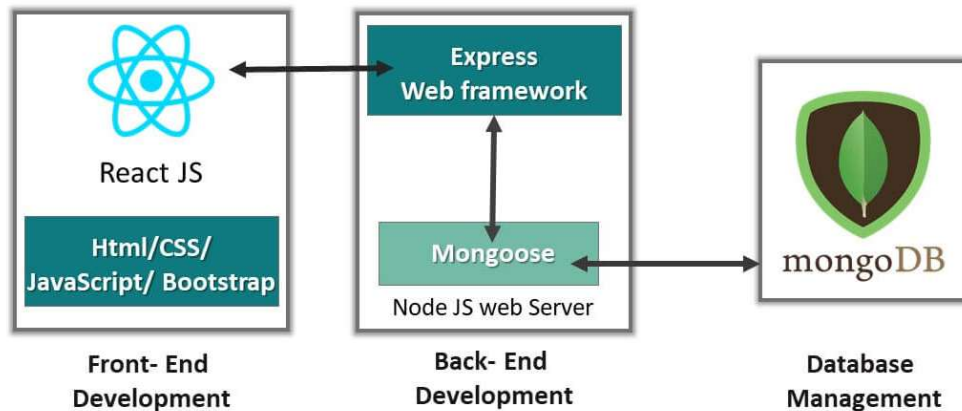


The project architecture

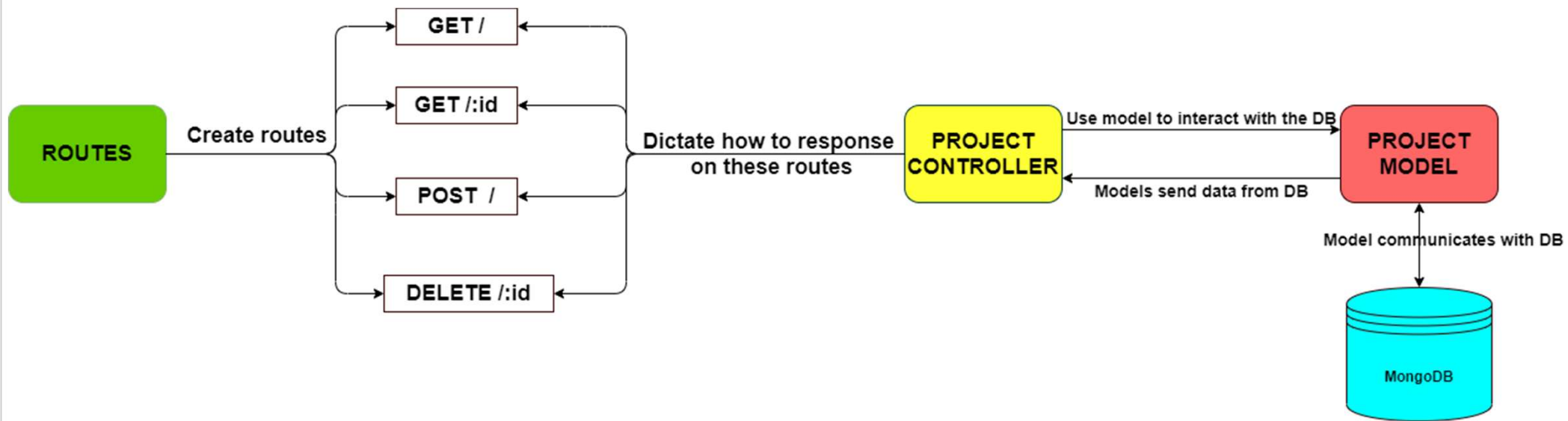
Our application will follow the standard REST architectural style: **React.js + Node.js + Express + MongoDB** application will follow this architecture:

You will have client and server components. The application will send client requests to our server. The server will then fetch the data from the MongoDB Atlas database and return it to the client

MERN Stack Development



The project architecture



Building RESTful APIs with Express.js

Express.js simplicity and flexibility make it a great option for creating RESTful APIs. We can build the RESTful API by doing the following steps:

- **Setup the Project:** Create a new Node.js project and install express.js.
- **Create Express App:** Require Express and create an instance using `express()`.
- **Define Routes:** Designate route handlers for HTTP methods (GET, POST, PUT, DELETE) and endpoints. These handlers will determine how the API responds to specific requests.
- **Middleware:** Implement middleware functions to handle tasks like authentication, request parsing, error handling, etc. Middleware functions are executed before reaching the route handlers.
- **Create Controllers:** Create controllers by separating the business logic from the route handlers. Controllers manage data processing and interact with databases or other services.
- **Connect to Database:** If your API requires data storage, connect to a database (e.g., MongoDB) using a suitable database driver.
- **Test API Endpoints:** Use tools like Postman or cURL to test the API endpoints and verify they return the expected responses.
- **Error Handling:** Implement error handling middleware to catch and manage errors gracefully.
- **Deploy the API:** Once your API is ready, deploy it to a server or cloud platform to make it accessible to clients.

Step 1: Set Up Your Development Environment

Install Node.js and npm

First, you'll need to download and install Node.js from <https://nodejs.org/>. Then you can verify the installation by running:

`node -v` and `npm -v`.

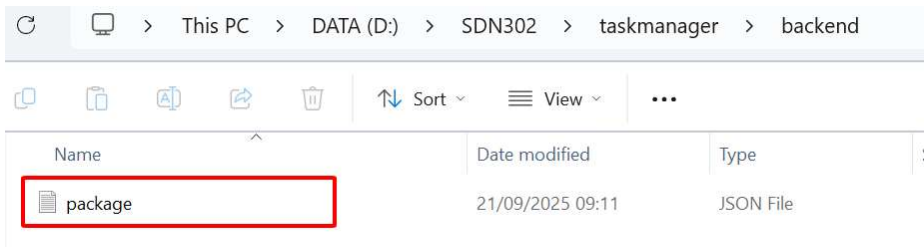
Download Node.js



Initialize a New Project

Create a new directory for your project. Then initialize the project with npm:

`npm init -y`



```
D:\SDN302\taskmanager\backend>npm init -y
Wrote to D:\SDN302\taskmanager\backend\package.json:

{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

Step 2: Set Up the Server

To set up your server, create a file called **server.js**

These are common names. They are used for their descriptive nature. But, you can name the file whatever you prefer.

The **server.js** file will be used to create a server which will be used to manage, control, and route to the necessary page in our application.

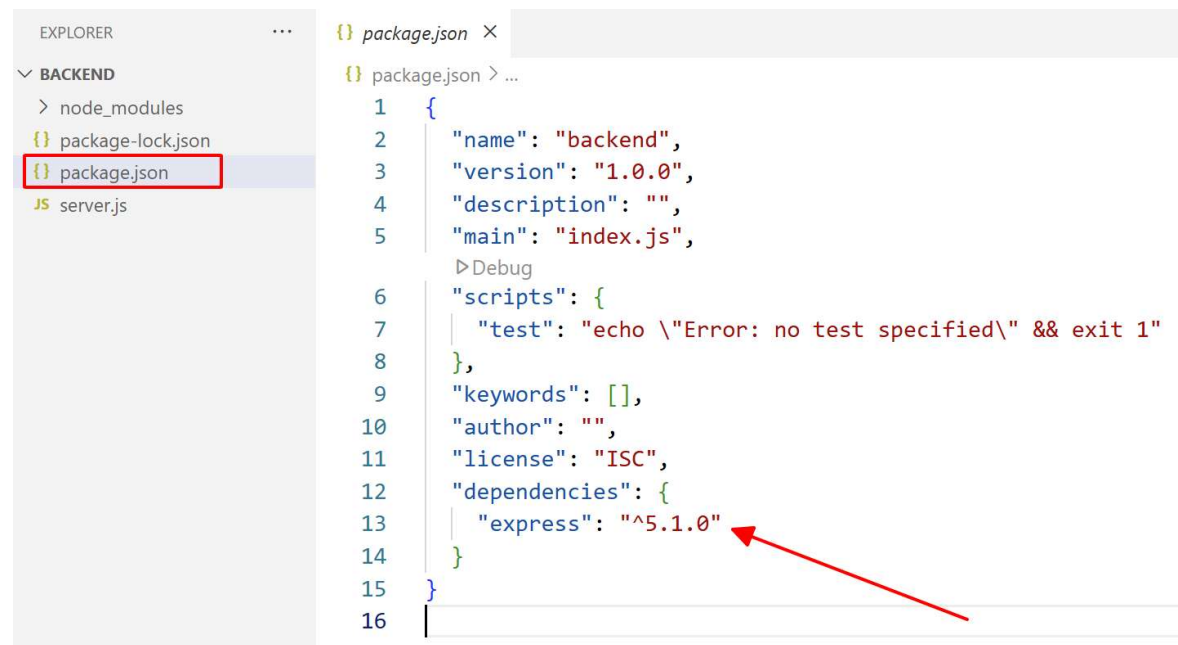
Install and Set Up Express.js

Express.js is a popular web application framework for Node.js and a third-party library that we use in our application.

Express simplifies the handling and definition of various routes for HTTP requests. It enables you to manage the application's routing and connect it to the server

Install Express.js by running the following command in your terminal:

```
npm install express
```



The screenshot shows a code editor with two panels. The left panel, titled 'EXPLORER', shows a file tree with a folder named 'BACKEND'. Inside 'BACKEND', there are three files: 'node_modules', 'package-lock.json', and 'package.json'. The 'package.json' file is highlighted with a red box. Below it is 'server.js'. The right panel shows the content of 'package.json'. The JSON object has the following properties: 'name' is 'backend', 'version' is '1.0.0', 'description' is an empty string, 'main' is 'index.js', 'scripts' is an object with a 'test' property set to 'echo \"Error: no test specified\" && exit 1', 'keywords' is an empty array, 'author' is an empty string, 'license' is 'ISC', and 'dependencies' is an object with 'express' set to '^5.1.0'. A red arrow points to the 'express' dependency value.

```
{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^5.1.0"
  }
}
```

Step 2: Set Up the Server

Creating a Simple Express Server: In the root of your project, create a file called **server.js**:

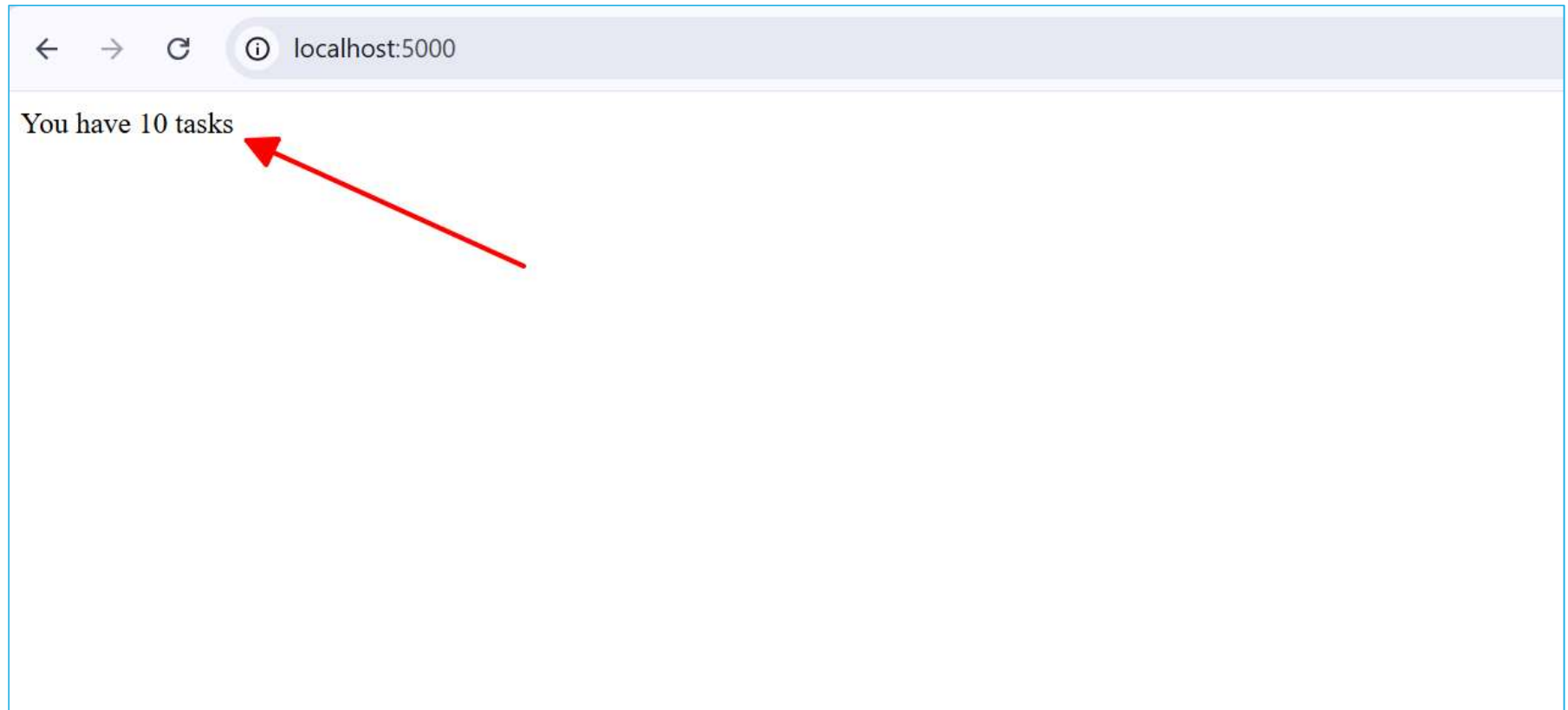
```
const express = require('express')
const app = express();

// middlewares config req.body
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

//get api
app.get('/', (req, res) => {
  res.status(200).send('You have 10 tasks');
});

const PORT = 5000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`)
})
```

TEST POSTMAN



Step 2: Set Up the Server

- ❑ To start the server, go to your terminal.

```
node --watch server.js
```

- ❑ Or you can install [nodemon] (<https://www.npmjs.com/package/nodemon>) for the same purpose. Nodemon detects changes in the directory and restarts your application.

```
npm install nodemon -D
```

- ❑ Then config in package.json:

```
"scripts": {  
  "dev": "nodemon app.js",  
  "start": "node app.js"  
}
```

- ❑ Then run: **npm run dev**

package.json

package.json > ...

```
1  {  
2    "name": "backend",  
3    "version": "1.0.0",  
4    "description": "",  
5    "main": "server.js",  
6    "type": "module",  
7    "scripts": {  
8      "dev": "nodemon server.js",  
9      "start": "node server.js"  
10   },  
11   "keywords": [],  
12   "author": "",  
13   "license": "ISC",  
14   "dependencies": {  
15     "express": "^5.1.0"  
16   },  
17   "devDependencies": {  
18     "nodemon": "^3.1.10"  
19   }  
20 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\SDN302\taskmanager\backend> npm run dev

Step 2: Set Up the Server

NOTE: package.json

"type": "module"

```
1 import express from 'express'
2
3 import {
4   createTask,
5   deleteTask,
6   getAllTasks,
7   updateTask
8 } from '../controllers/tasksControllers.js';
9
10 const router = express.Router()
11
12 // Define the routes for this router
13 router.get('/', getAllTasks);
14
15 router.post('/', createTask);
16
17 router.put('/:id', updateTask)
18
19 router.delete('/:id', deleteTask)
20
21 export default router
```

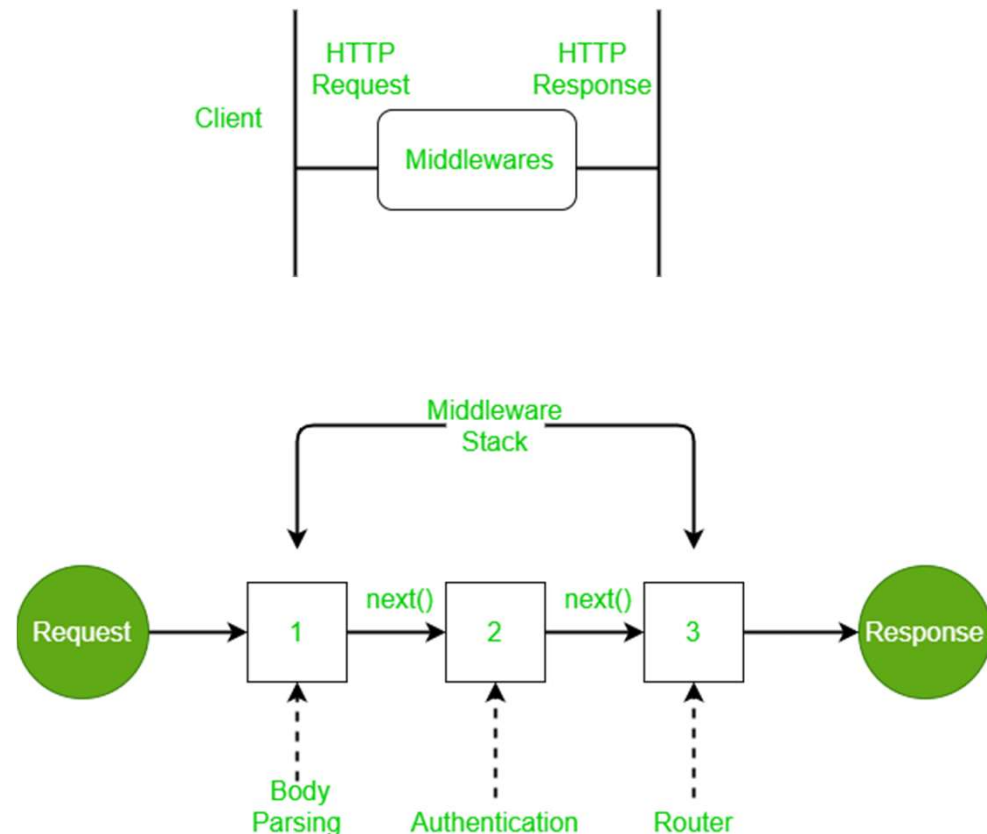
```
1 const express = require('express');
2
3 const ctrl = require('../controllers/tasksControllers')
4
5 const router = express.Router();
6
7 // Define the routes for this router
8 router.get('/', ctrl.getAllTasks);
9
10 router.post('/', ctrl.createTask);
11
12 router.put('/:id', ctrl.updateTask)
13
14 router.delete('/:id', ctrl.deleteTask)
15
16 module.exports = router;
```

Step 2: Set Up the Server

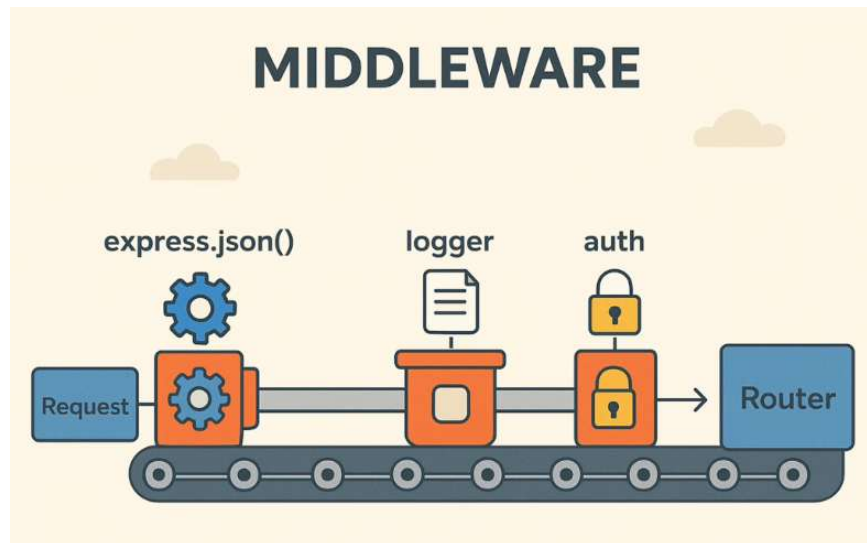
Middleware in Express

Middleware functions in Express.js can perform several important tasks:

- 1. Execute Code:** Middleware can run any code when a request is received.
- 2. Modify Request and Response:** Middleware can modify both the request (req) and response (res) objects.
- 3. End the Request-Response Cycle:** Middleware can send a response to the client, ending the cycle.
- 4. Call the Next Middleware:** Middleware can call `next()` to pass control to the next function in the middleware stack.



Step 2: Set Up the Server



Then in your **server.js** file, register middleware in Express using **app.use(...)**. Middleware is an intermediary function that runs before the request reaches the route handler.

```
// middlewares
app.use(express.json());

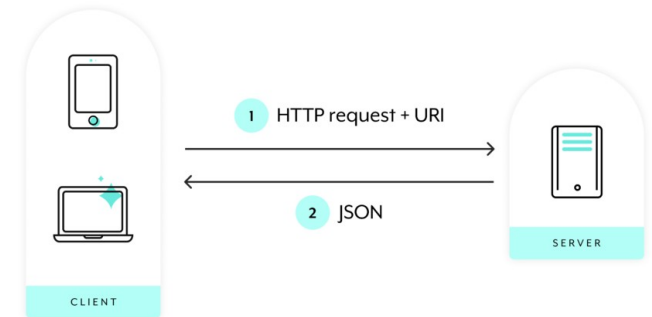
// Parse form-urlencoded (form HTML)
app.use(express.urlencoded({ extended: true }));
```

Step 2: Set Up the Server

❑ Create CRUD operations API in server.js:

```
const express = require('express')
const app = express();

// middlewares
app.use(express.json());
// Parse form-urlencoded (form HTML)
app.use(express.urlencoded({ extended: true }));
// routing path
app.get('/', (req, res) => {
  res.status(200).send('You have 10 tasks');
});
app.post('/api/tasks', (req, res) => {
  res.status(201).json({ message: 'Task added successfully!' })
});
app.put('/api/tasks/:id', (req, res) => {
  res.status(204).json({ message: 'Task updated successfully!' })
})
app.delete('/api/tasks/:id', (req, res) => {
  res.status(200).json({ message: 'Task delete successfully!' })
})
const PORT = 5000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`)
})
```

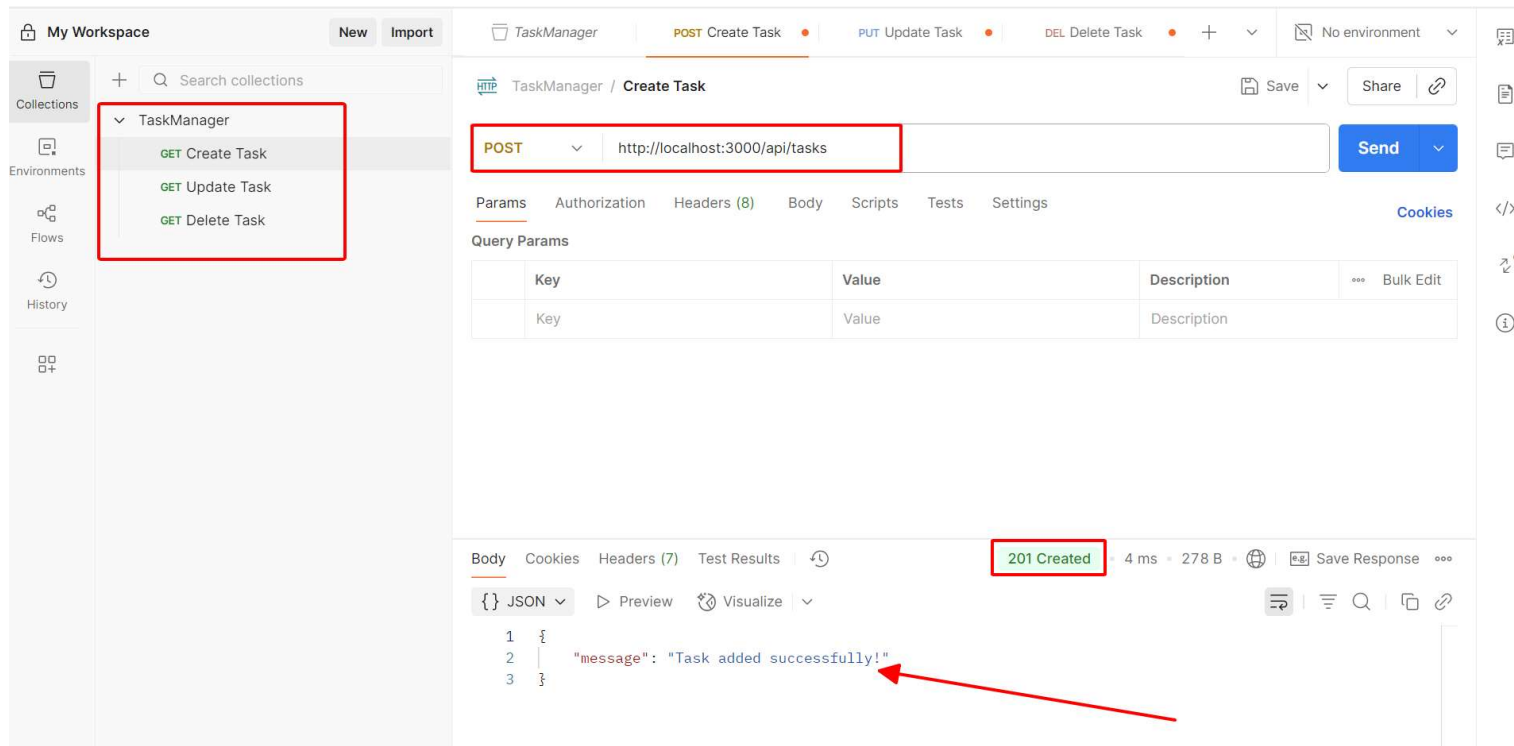


HTTP Status Codes		
Level 200	Level 400	Level 500
200: OK 201: Created 202: Accepted 203: Non-Authoritative Information 204: No content	400: Bad Request 401: Unauthorized 403: Forbidden 404: Not Found 409: Conflict	500: Internal Server Error 501: Not Implemented 502: Bad Gateway 503: Service Unavailable 504: Gateway Timeout 599: Network Timeout

Step 2: Set Up the Server

❑ Test API with Postman:

Postman is a API testing tool used by more than 20 million users. The simple and user-friendly interface helps when it comes to documenting, designing, and testing APIs. You can create requests to API endpoints, send various types of data, and examine responses effortlessly with Postman. Its simple and informative graphical user interface makes it easy for even a less experienced beginner to interact with APIs without digging into complex code



Step 3: Define Routes

❑ Using `express.Router()`:

In a real application where we could have +100 endpoints our `server.js` would be a really mess.

To avoid that we will structure our app in Models/Controllers/Routes. In the most informal way these will mean.

Organize the application into **src/routes** folders

▼ BACKEND

> node_modules

▼ src

> config

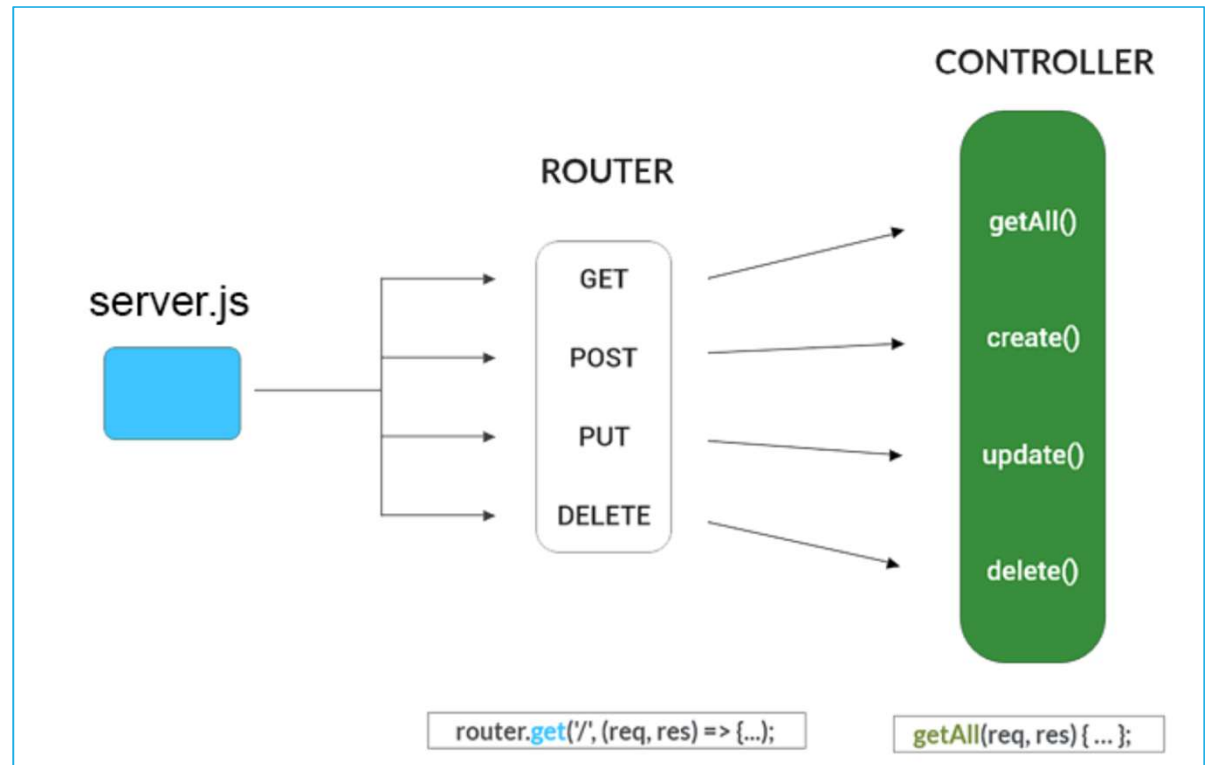
> controllers

> models

▼ routes

JS tasksRoutes.js

JS usersRoutes.js



Step 3: Define Routes

❑ Using `express.Router()`:

Creating a Router: for the `/tasks` route.
(e.g., `taskRoutes.js`):

```
const express = require('express')
const router = express.Router();

// routing path
router.get('/', (req, res) => {
  res.status(200).send('You have 10 tasks');
});
router.post('/', (req, res) => {
  res.status(201).json({ message: 'Task added successfully!' });
});
router.put('/:id', (req, res) => {
  res.status(204).json({ message: 'Task updated successfully!' });
});
router.delete('/:id', (req, res) => {
  res.status(200).json({ message: 'Task delete successfully!' });
});

module.exports = router;
```

In your main `server.js`, you'll now import the router and tell your main app to use it:

```
const express = require('express');
const tasksRoutes =
  require('./src/routes/taskRoutes');
const app = express();

// middlewares
app.use(express.json());

// Parse form-urlencoded (form HTML)
app.use(express.urlencoded({ extended: true }));

app.use('/api/tasks', tasksRoutes);

const PORT = 5000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`)
})
```

Step 4: Model/View/Controller structure

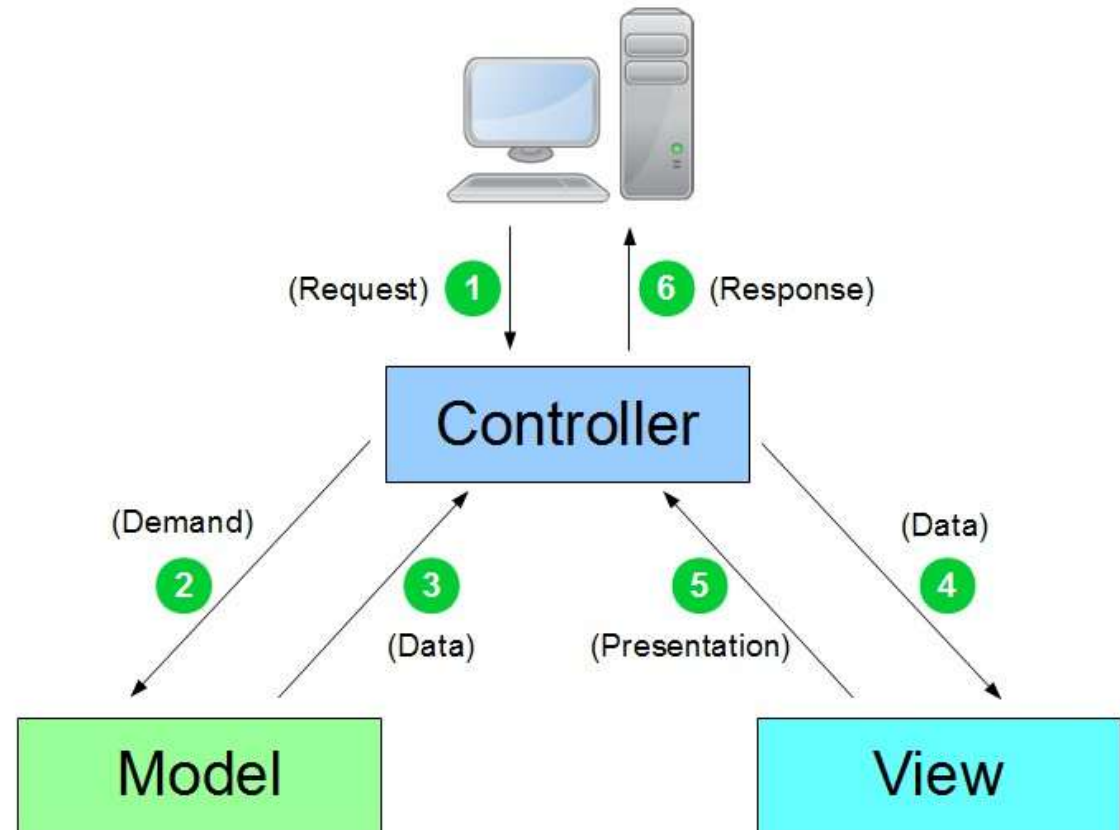
MVC Pattern with Node.js

The MVC pattern divides an application into three distinct parts:

1.Model: Represents the data and business logic of the application. It interacts with the database and processes data independently of the user interface.

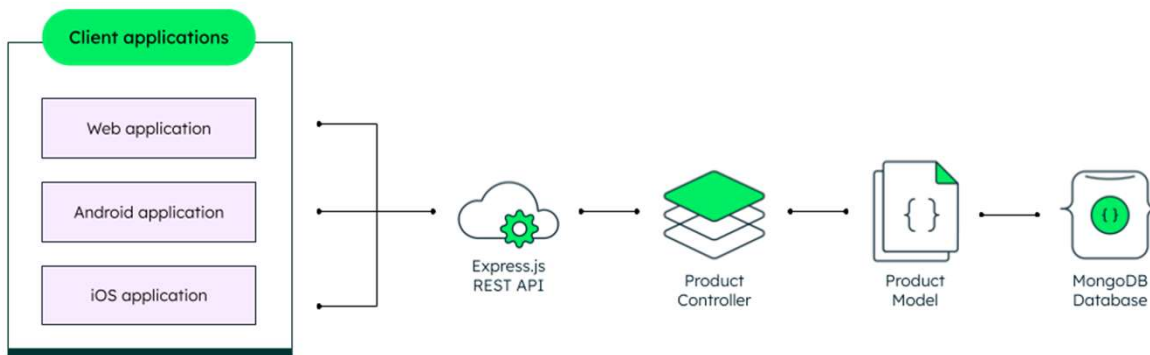
2.View: Handles the presentation layer of the application. It displays data to the user and sends user commands to the Controller.

3.Controller: Acts as an intermediary between the Model and the View. It takes user input, interacts with the Model, and updates the View accordingly.



Step 4: Model/Router/Controller structure

Organize the application into models, views, and controllers folders. This structure is essential for an MVC pattern



```
> node_modules
✓ src
  > config
  ✓ controllers
    JS taskController.js
  > models
  > routes
  ⚙ .env
  💎 .gitignore
  {} package-lock.json
  {} package.json
  JS server.js
```

Step 4: Model/View/Controller structure

Create a **taskController.js** file inside **controllers** folder
The Controller handles the application logic, processes user inputs, interacts with the Model, and directs the View to render the data

```
// controllers/taskController.js

exports.getAllTasks = (req, res) => {
  res.status(200).send('You have 10 tasks');
};

exports.createTasks = (req, res) => {
  res.status(201).json({ message: 'Task added successfully!' });
};

exports.updateTasks = (req, res) => {
  res.status(204).json({ message: 'Task updated successfully!' });
};

exports.deleteTasks = (req, res) => {
  res.status(200).json({ message: 'Task delete successfully!' });
};
```

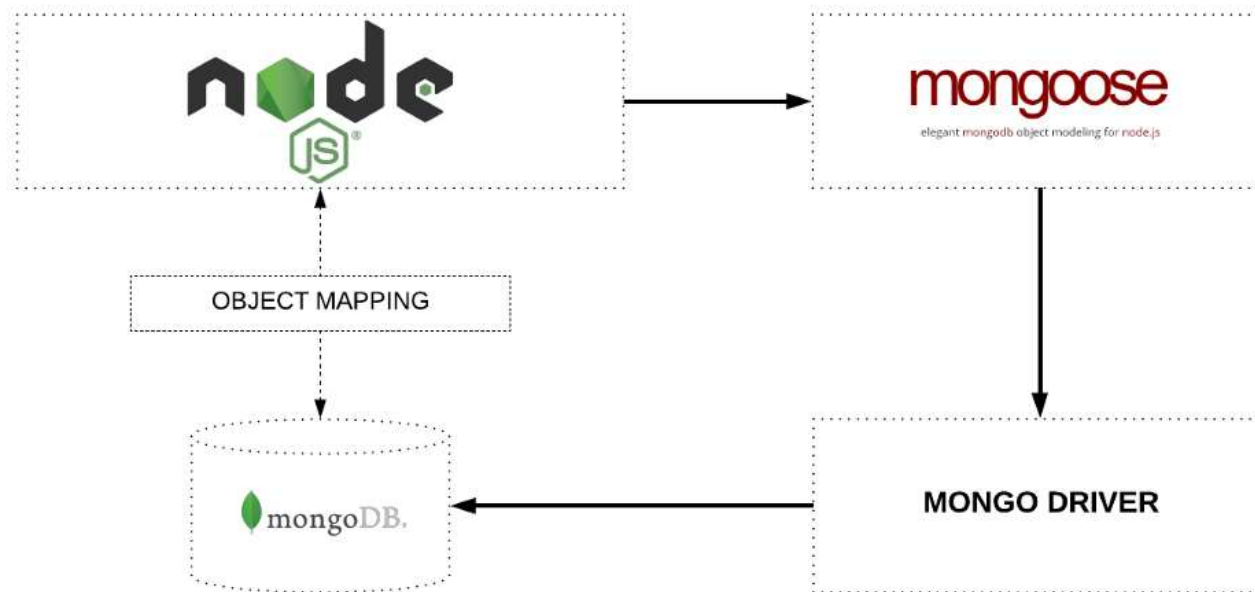
In your main **taskRoutes.js**, you'll now import the taskController and use it:

```
src > routes > JS taskRoutes.js > ...
1  const express = require('express');
2
3  const ctrl = require('../controllers/taskController');
4
5  const router = express.Router();
6
7  // routing path
8  router.get('/', ctrl.getAllTasks);
9  router.post('/', ctrl.createTasks);
10 router.put('/:id', ctrl.updateTasks);
11 router.delete('/:id', ctrl.deleteTasks);
12
13 module.exports = router;
```

Step 5: Connect to the Database

Mongoose is a JavaScript library that makes it easier to work with MongoDB in a Node.js environment. It provides a simple way to model your data. You can define schemas, do data validation, and build queries as well. Install Mongoose using npm.

```
npm i mongoose
```



MongoDB Compass

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

huanll

admin

config

local

taskmanager

tasks

taskmanager

huanll > taskmanager

Open MongoDB shell

Create collection

Refresh

Collection name	Properties	Storage size	Documents	Avg. document size	Indexes	Total index size
tasks	-	4.10 kB	0	0 B	1	4.10 kB

New Connection

Manage your connection settings

URI ? Edit Connection String 🔴

mongodb://localhost:27017/

How do I find my connection string in Atlas?

If you have an Atlas cluster, go to the Cluster view. Click the

Step 5: Connect to the Database

Create a **db.js** file inside **config** folder. You need to define the connection URL provided in your db.js file:

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect('mongodb://127.0.0.1:27017/taskmanager')
    console.log('Connection successful')
  } catch (error) {
    console.error("Error connecting to database:", error);
    process.exit(1); // exit with error
  }
}

module.exports = connectDB;
```

In your **server.js** file, you need to require Mongoose to use it as a connector to the database.

```
const connectDB = require('./src/config/db')

// Connect to the Database
connectDB();
```

Step 6: Set up your .env and .gitignore files

we'll upload the project to GitHub. You'll protect your port number and MongoDB URL with secure storage. You'll also use an environment variable file, a configuration file known as .env. This file keeps sensitive information safe. It includes passwords and API URLs and prevents exposure. Here's how to set it up using Node.js:

```
npm i dotenv
```

Then create a .env file. Inside it, add your PORT number and MongoDB URL. It should look something like this:

```
PORT=5000  
MONGODB_URI=mongodb://127.0.0.1:27017/taskmanager
```

Then update your .gitignore file:

```
/node_modules  
.env
```

Then in your server.js file, require the dotenv package. Load the variables with this line at the top of the file:

```
const dotenv = require('dotenv');  
dotenv.config();
```

Step 6: Set up your .env and .gitignore files

Then, in your Node.js code, you can access these variables using the `process.env` object:

In the `db.js` file edit the codes below:

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI)
    console.log('Connection successful')
  } catch (error) {
    console.error("Error connecting to database:", error);
    process.exit(1); // exit with error
  }
}

module.exports = connectDB;
```

In the `server.js` file edit the codes below:

```
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server is running on
http://localhost:${PORT}`)
})
```

Step 7: Create the Model for the Document Structure

Define schemas using Mongoose to model the structure of your MongoDB documents. Schemas allow you to define your data's fields, types, and validations. Use the **Mongoose.Schema()** method to create a new schema, specifying the fields and their types. Schema is the blueprint for how the data is structured

To create this model:

Create a folder named **models** to keep your application organized.

Inside the models folder, create a file called **Task.js**

Task (model)

```
{
  title: "học react",
  status: 'active',
  completedAt: "2025-08-24 09:00",
  createdAt: "2025-08-24 00:00",
  updatedAt: "2025-08-24 01:00"
}
```

Task (schema)

```
{
  title: {
    type: String,
    required: true
  },
  status: {
    type: String
  },
  ...
}
```

```
> node_modules
✓ src
  > config
  > controllers
  ✓ models
    JS Task.js
  > routes
  .env
  .gitignore
  {} package-lock.json
  {} package.json
  JS server.js
```

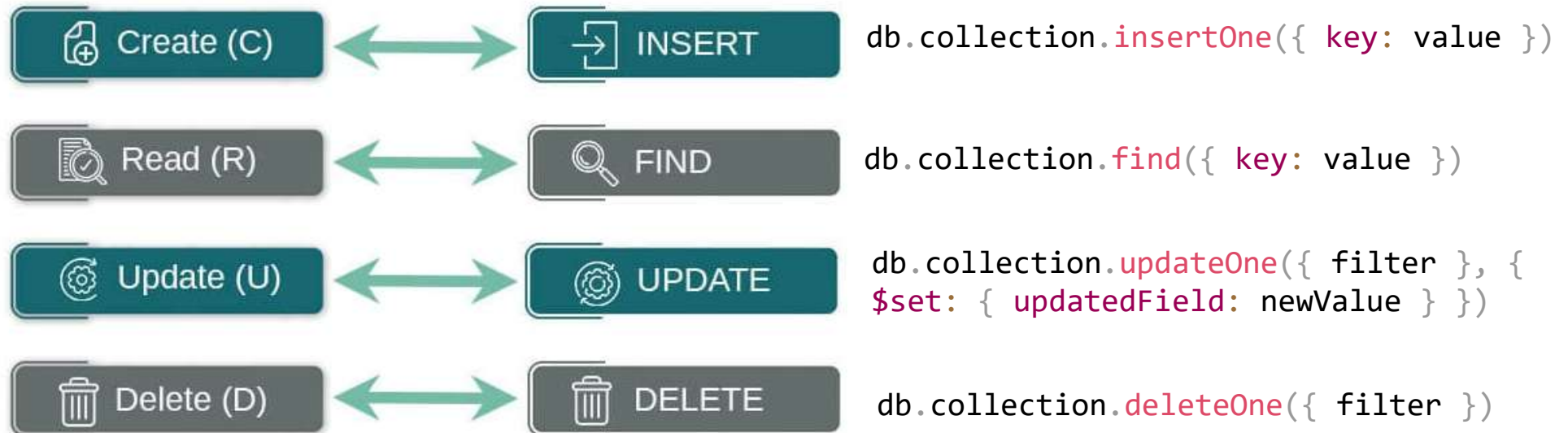
Step 7: Create the Model for the Document Structure

```
const mongoose = require('mongoose');
const taskSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: true,
      trim: true,
    },
    status: {
      type: String,
      enum: ["active", "complete"],
      default: "active",
    },
    completedAt: {
      type: Date,
      default: null,
    }
  },
  {
    timestamps: true, // createdAt and updatedAt are automatically added
  }
);

const Task = mongoose.model("Task", taskSchema);
module.exports = Task;
```

Step 8: CRUD operations in MongoDB

CRUD stands for **Create**, **Read**, **Update**, and **Delete**, the basic operations for manipulating data in a database. MongoDB provides a set of operations to perform CRUD operations on documents within collections. Let's see one by one:



Step 8: CRUD operations in MongoDB

In the `taskController.js` file edit the codes below:

```
const Task = require('../models/Task');

// controllers/taskController.js
exports.getAllTasks = async (req, res) => {
  try {
    const tasks = await Task.find();
    res.status(200).json(tasks);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

exports.createTasks = async (req, res) => {
  try {
    const task = new Task(req.body);
    await task.save();
    res.status(201).json(task);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
};
```

Step 8: CRUD operations in MongoDB

In the `taskController.js` file edit the codes below:

```
exports.updateTasks = async (req, res) => {
  try {
    const { title, status, completedAt } = req.body;
    const updatedTask = await Task.findByIdAndUpdate(
      req.params.id, {
        title,
        status,
        completedAt
      },
      { new: true }
    );
    if (!updatedTask) {
      return res.status(404).json({ message: "The task does not exist."
    });
  }
  res.status(200).json(updatedTask);
} catch (error) {
  res.status(500).json({ error: error.message });
}
};
```

```
exports.deleteTasks = async (req, res) => {
  try {
    const deleteTask = await
    Task.findByIdAndDelete(req.params.id);
    if (!deleteTask) {
      return res.status(404).json({
        message: "The task does not exist." });
    }
    res.status(200).json(deleteTask);
  } catch (error) {
    res.status(500).json({ error:
      error.message });
  }
};
```

Step 9: Test API Endpoint

1. Test getAllTasks → Result: Empty

The screenshot shows a REST client interface with a sidebar on the left listing collections. The 'TaskManager' collection is expanded, and the 'Get All Task' endpoint is selected. The main panel shows the request details for a GET request to 'http://localhost:5000/api/tasks'. The response is a 200 OK status with a 6 ms response time and 235 B of data. The response body is shown as a JSON array '[]', which is highlighted with a red arrow and the word 'Empty'.

Search collections

- fitnessapp_studentcode
 - TaskManager
 - POST Create Task
 - GET Update Task
 - GET Delete Task
 - GET Get All Task
 - POST Create User
 - GET Get User
 - POST Login
 - GET Get Role
 - GET Test home
 - Todo-api

TaskManager / Get All Task

GET http://localhost:5000/api/tasks

Send

Docs Params Auth Headers (7) Body Scripts Tests Settings Cookies

Query Params

	Key	Value	Description	⋮ Bulk Edit
	Key	Value	Description	

Body 200 OK • 6 ms • 235 B • Save Response

{ } JSON Preview Visualize

1 []

Empty

Step 9: Test API Endpoint

2. Test createTasks

The screenshot displays the Postman interface for testing an API endpoint. On the left, a sidebar lists collections, with 'TaskManager' expanded and 'POST Create Task' selected. The main workspace shows a POST request to 'http://localhost:5000/api/tasks'. The request body is a JSON object:

```
{  "title": "Task 1"}
```

. The response status is '201 Created', with a response time of 63 ms and a size of 415 B. The response body is a detailed JSON object:

```
{  "title": "Task 1",  "status": "active",  "completedAt": null,  "_id": "696db8a47a6283819840b948",  "createdAt": "2026-01-19T04:52:52.462Z",  "updatedAt": "2026-01-19T04:52:52.462Z",  "__v": 0}
```

. A red arrow points to the '201 Created' status.

Search collections

fitnessapp_studentcode

TaskManager

- POST Create Task
- GET Update Task
- GET Delete Task
- GET Get All Task
- POST Create User
- GET Get User
- POST Login
- GET Get Role
- POST Test home

Todo-api

HTTP TaskManager / Create Task

POST <http://localhost:5000/api/tasks>

Send

Docs Params Auth Headers (9) Body Scripts Tests Settings

raw JSON

```
1 {
2   "title": "Task 1"
3 }
```

Schema Beautify

Body 201 Created 63 ms 415 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "title": "Task 1",
3   "status": "active",
4   "completedAt": null,
5   "_id": "696db8a47a6283819840b948",
6   "createdAt": "2026-01-19T04:52:52.462Z",
7   "updatedAt": "2026-01-19T04:52:52.462Z",
8   "__v": 0
9 }
```

Step 9: Test API Endpoint

3. Test getAllTasks → Result: show Task 1

The screenshot shows the Postman interface for testing the `GET /api/tasks` endpoint. The left sidebar lists collections, with `TaskManager` expanded and `GET Get All Task` selected. The main area shows the request details: `GET` method and `http://localhost:5000/api/tasks` URL. The response is a `200 OK` with a status of `8 ms` and `412 B`. The response body is a JSON array containing one task object.

Request:

```
GET http://localhost:5000/api/tasks
```

Response:

```
200 OK • 8 ms • 412 B • Save Response
```

```
{
  "_id": "696db8a47a6283819840b948",
  "title": "Task 1",
  "status": "active",
  "completedAt": null,
  "createdAt": "2026-01-19T04:52:52.462Z",
  "updatedAt": "2026-01-19T04:52:52.462Z",
  "__v": 0
}
```

A red arrow points to the response body, highlighting the task object.

Step 9: Test API Endpoint

4. Test updateTasks → The ID is in the correct format but does NOT exist in the database.

The screenshot shows a REST client interface with a sidebar on the left containing a collection of API endpoints. The 'TaskManager' collection is expanded, and the 'Update Task' endpoint (GET) is selected and highlighted with a red box. The main panel displays the details for the 'Update Task' endpoint, showing a PUT request to the URL 'http://localhost:5000/api/tasks/696db8a47a6283819840b941'. The request body is a JSON object:

```
{  "title": "Task 1-edited"}
```

. The response status is '404 Not Found' with a message:

```
{  "message": "The task does not exist."}
```

. A red arrow points to the response message. The interface includes tabs for Docs, Params, Auth, Headers (9), Body, Scripts, Tests, and Settings. The response body is shown in JSON format, and the status bar indicates a 404 Not Found error.

TaskManager / Update Task

PUT http://localhost:5000/api/tasks/696db8a47a6283819840b941

raw JSON

```
1 {
2   "title": "Task 1-edited"
3 }
```

Body 404 Not Found • 41 ms • 280 B • Save Response

{ } JSON Preview Debug with AI

```
1 {
2   "message": "The task does not exist."
3 }
```

Step 9: Test API Endpoint

5. Test updateTasks → Result: edit Task 1 to Task 1-edited

The screenshot shows the Postman interface for testing an API endpoint. On the left, a collection named 'TaskManager' is expanded, and the 'Update Task' endpoint (GET) is selected. The main panel shows the request configuration for the 'Update Task' endpoint, which is a PUT request to 'http://localhost:5000/api/tasks/696db8a47a6283819840b948'. The request body is a JSON object with the title 'Task 1-edited'. The response is a 200 OK status with a JSON body containing task details, including the updated title 'Task 1-edited'.

Request Configuration:

- Method: PUT
- URL: http://localhost:5000/api/tasks/696db8a47a6283819840b948
- Body:

```
{
  "title": "Task 1-edited"
}
```

Response:

```
{
  "_id": "696db8a47a6283819840b948",
  "title": "Task 1-edited",
  "status": "active",
  "completedAt": null,
  "createdAt": "2026-01-19T04:52:52.462Z",
  "updatedAt": "2026-01-19T06:01:04.107Z",
  "__v": 0
}
```

Step 9: Test API Endpoint

6. Test deleteTasks → The ID is in the correct format but does NOT exist in the database.

The screenshot shows a REST client interface with a sidebar on the left listing collections. The 'TaskManager' collection is expanded, and the 'Delete Task' endpoint is selected. The main panel shows a DELETE request to `http://localhost:5000/api/tasks/696db8a47a6283819840b941`. The response is a 404 Not Found status with a message: `"message": "The task does not exist."`. A red arrow points to the message in the response body.

TaskManager / Delete Task

DELETE `http://localhost:5000/api/tasks/696db8a47a6283819840b941` **Send**

Docs Params Auth Headers (7) Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body **404 Not Found** • 7 ms • 280 B • Save Response

```
{
  "message": "The task does not exist."
}
```

Step 9: Test API Endpoint

7. Test deleteTasks → The ID is in the correct format and exist in the database.

The screenshot displays a REST client interface with a sidebar on the left containing a collection of API endpoints. The 'Delete Task' endpoint is highlighted. The main panel shows the details of a DELETE request to 'http://localhost:5000/api/tasks/696db8a47a6283819840b948'. The response is a 200 OK status with a response time of 48 ms and a body size of 417 B. The response body is shown in JSON format, containing task details such as '_id', 'title', 'status', 'completedAt', 'createdAt', 'updatedAt', and '__v'.

TaskManager / Delete Task

DELETE Send

Docs Params Auth Headers (7) Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description
Key	Value	Description

Body 200 OK • 48 ms • 417 B • Save Response

```
{
  "_id": "696db8a47a6283819840b948",
  "title": "Task 1-edited",
  "status": "active",
  "completedAt": null,
  "createdAt": "2026-01-19T04:52:52.462Z",
  "updatedAt": "2026-01-19T06:01:04.107Z",
  "__v": 0
}
```

Step 9: Test API Endpoint

8. Test getAllTasks → Result: Empty

The screenshot shows a REST client interface with a sidebar on the left containing a search bar and a list of collections. The 'TaskManager' collection is expanded, showing several endpoints. The 'GET Get All Task' endpoint is selected and highlighted with a red box. The main panel shows the details of this endpoint, including the method 'GET' and the URL 'http://localhost:5000/api/tasks', which are also highlighted with a red box. Below the URL bar are tabs for 'Docs', 'Params', 'Auth', 'Headers (7)', 'Body', 'Scripts', 'Tests', and 'Settings'. The 'Params' tab is active, showing a table for 'Query Params' with columns 'Key', 'Value', and 'Description'. The response section at the bottom shows a status of '200 OK' and a body of '[]' (an empty array). A red arrow points from the word 'Empty' to the '[]' in the response body.

Search collections

- fitnessapp_studentcode
 - TaskManager
 - POST Create Task
 - GET Update Task
 - GET Delete Task
 - GET Get All Task**
 - POST Create User
 - GET Get User
 - POST Login
 - GET Get Role
 - GET Test home
 - Todo-api

TaskManager / Get All Task

GET Send

Docs Params Auth Headers (7) Body Scripts Tests Settings Cookies

Query Params

	Key	Value	Description	⋮ Bulk Edit
	Key	Value	Description	

Body 6 ms • 235 B • Save Response ⋮

JSON

1 **Empty**